A Recipe-Based Model-Management Architecture for Versioned, Reproducible Training

Johnny Morgan
Information Systems, UMBC
yj07538@umbc.edu

***Abstract*—We present a model-management architecture in which every trained model is reproducible because every input to its training is a versioned, re-executable artifact. Data acquisition is expressed as *recipes*—recorded queries with time windows, snapshot dates, and join specifications against typed data sources (metadata, content, requirements). Labels are produced by versioned *label rules* derived from a deterministic requirements repository, with human adjudications layered as versioned *correction overlays* that never mutate the system of record. Models—trunks, classification heads, and baselines—are filesystem artifacts registered in a database with model cards, storage locations, hyperparameters, metrics, and a parent-model reference that makes fine-tuning lineage a directed acyclic graph. Orchestrated retraining at defined cadences (monthly head refresh, annual trunk fine-tuning) executes as recorded jobs, so reproducibility is a property of the system—re-execute the recipe—rather than a discipline asked of its operators. The architecture governs the frozen-trunk multi-customer classifier described in a companion paper [1] and generalizes to any model program requiring auditable, versioned training.**

I. Introduction

Model programs fail reproducibility not at training time but at data time: the model file survives while the query that built its training set, the requirements version that labeled it, and the corrections humans applied are lost. This architecture makes those inputs first-class artifacts. The organizing principle:

> *Reproducibility means the recipe re-executes—not that someone kept a copy of the dataframe.*

Two storage roles divide cleanly: the **filesystem** holds every large artifact (datasets, embedding caches, model weights, model cards); the **database** is the spine—it holds identity, versions, locations, parameters, and every relationship between artifacts. Nothing is known to the platform unless it is a row; nothing heavy lives in a row.

II. Data Source Abstraction

Sources are typed by role, not by technology:

- **Metadata source (A):** defines populations and supplies join keys; pulls are windowed in time.
- **Content source (B):** supplies the payload a model consumes (text here), joined by key.
- **Requirements source (C):** the label authority; deterministic rules over C produce labels.

The typing is what generalizes: any program with a population source, a payload source, and a labeling authority maps onto the same schema.

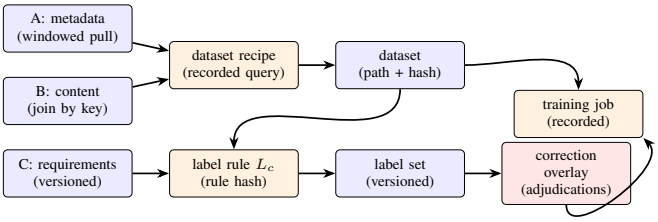

Fig. 1. Acquisition as versioned artifacts: recipes materialize datasets; label rules over C produce label sets; adjudications overlay without mutating the system of record; training jobs record exactly which versions they consumed.

III. Recipes, Label Rules, and Correction Overlays

Fig. 1 shows acquisition. A **dataset recipe** records the A query, time window, snapshot date, and B join specification; executing it materializes a dataset whose storage path and corpus hash are registered. A **label rule** records the C requirements reference and version per customer plus a rule hash; applying it to a dataset yields a **label set**, versioned and registered with class balance. Human adjudications (from the companion system's disagreement queue [1]) are written to a **correction overlay**—a versioned table of per-record label amendments with adjudicator, timestamp, and the model version that surfaced the record. C is never mutated; the effective label is (label set $\oplus$ overlay), and every training job records which overlay version it consumed.

IV. Registry Schema

```
data_sources(id, name, role[A|B|C], connection_ref)
dataset_recipes(id, a_query, time_window,
    snapshot_date, b_join_spec, created_at)
datasets(id, recipe_id, storage_path,
    n_records, corpus_hash)
label_rules(id, customer_id, c_requirements_ref,
    requirements_version, rule_hash)
label_sets(id, dataset_id, label_rule_id,
    version, storage_path, class_balance)
correction_overlays(id, label_set_id, version,
    storage_path)
corrections(overlay_id, record_id, old_label,
    new_label, adjudicator, source_model_id, at)
feature_sets(id, version, extraction_spec,
    storage_path)
models(id, kind[trunk|head|baseline], name,
    version, parent_model_id, dataset_id,
    label_set_id, overlay_version,
    feature_set_id, trunk_model_id,
    storage_path, model_card_path,
    hyperparams_json, metrics_json,
    threshold, trained_at)
```

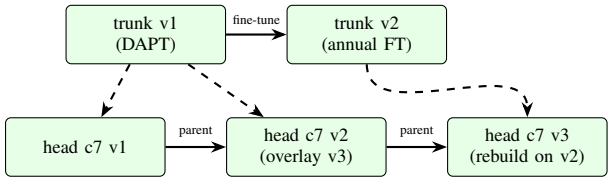

Fig. 2. Lineage DAG. Solid edges: parent (fine-tune/warm-start). Dashed: trunk dependency. Every node carries its dataset, label-set, and overlay versions.

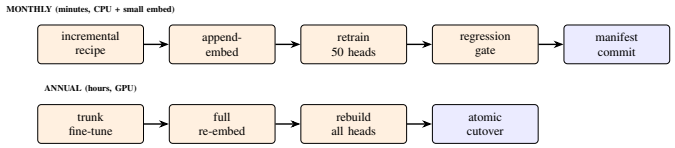

Fig. 3. Cadence as managed job chains. Monthly holds minimum viable product; annual refreshes the representation.

```
training_jobs(id, model_id, cadence
    [monthly|annual|on_demand], recipe_id,
    status, started_at, finished_at, log_path)
manifest(deployed_at, trunk_model_id,
    head_model_ids_json, committed_by)
```

Every column that names an artifact stores a filesystem path; every relationship is a foreign key. The model card is a human-readable file (scope, intended use, data description, metrics, caveats) whose location—not content—lives in the row.

V. Model Lineage

`parent_model_id` makes model history a DAG (Fig. 2): a trunk fine-tuned from its predecessor, heads warm-started from prior heads, all heads referencing the trunk (and feature-set version) they consume. Lineage answers the audit question in one query: for any deployed prediction, which head version, trained on which dataset and label versions including which adjudications, descended from which parents, against which trunk.

VI. Orchestrated Cadence

Fig. 3 shows the two loops from the companion system [1], expressed as managed jobs.

Monthly (head refresh). Materialize the month's incremental recipe; append-embed new records; apply current label rules and overlays; retrain all heads; recalibrate thresholds; run the regression gate (each head's metrics vs. its prior version); commit a new manifest. Every step writes rows; the whole month is one auditable job chain.

Annual (trunk refresh). Fine-tune or re-DAPT the trunk on the grown corpus (recorded as a job with `parent_model_id`); re-embed the full corpus; rebuild all heads; atomic manifest cutover. On-demand jobs (a requirement change in C, a batch of adjudications) run the same machinery on one head.

Rollback at any cadence is a manifest revert: artifacts are immutable, versions only accumulate.

VII. Reproducibility Guarantees

For any registered model: (1) its dataset is reconstructable by recipe re-execution against the recorded snapshot; (2) its labels are reconstructable from the label rule version plus overlay version; (3) its training is re-runnable from recorded hyperparameters and parent model; (4) its deployment history is the git log of the manifest. The guarantee is structural—enforced by foreign keys and immutable artifacts—not procedural.

VIII. Discussion

The architecture's generality follows from its abstractions: typed sources, recipes, rules, overlays, and a lineage DAG describe any supervised program whose labels derive from an authoritative repository and whose corrections come from humans. The companion classifier [1] is its first tenant; subsequent model programs join by adding rows, not systems.

References

[1] J. Morgan, "A Frozen-Trunk Multi-Head Architecture for Citation-Fidelity Remediation via Positive-Unlabeled Learning," companion paper, 2026.